

AI 技术应用场景匹配报告

基于《空箱》Unity 项目的软件过程定义，识别 AI 技术可显著提升效率的关键环节

1. 引言：AI 在软件工程领域的发展趋势（2024-2026）

2024 年至 2026 年，生成式 AI 与大语言模型（LLM）在软件工程领域经历了从「辅助工具」到「开发范式变革」的跃迁。根据中国信通院 2024 年度《AI4SE 行业现状调查报告》（联合 30 余家头部机构发布，有效问卷 1,813 份），65.75% 的企业已进入软件研发智能化的 L2-L4 阶段，代码生成平均采纳率达 27.46%，AI 生成的代码占比提升至 28.17%，测试环节的 AI 提效幅度最大（效率涨幅同比 +8 个百分点）[1]。

在学术研究层面，Cheng 等人（2025）在 *Software: Practice and Experience* 期刊上发表了首个聚焦生成式 AI 在需求工程中应用的系统文献综述，分析了 238 篇文献后发现：GPT 系列模型占研究的 67.3%，但超过 90% 的研究仍处于原型阶段，产业级落地面临可复现性（66.8%）、幻觉（63.4%）和可解释性（57.1%）三重核心挑战[2]。Science China Information Sciences（2025）发表的 LLM 在软件工程中的综述则从全生命周期视角梳理了 LLM 在需求、设计、编码、测试、维护各阶段的应用进展，指出 LLM 辅助开发可将任务完成速度提升 55.8%[3]。

在架构设计领域，Tagliaferro 等人（2025）在 IEEE ICSE 会议上发表了关于 LLM 从非正式规格自动生成软件架构设计的研究，首次提出量化指标评估 LLM 生成的 UML 组件图质量，结论是 LLM 方案有前景但尚未达到生产级精度[4]。Ferrari 等人（2024）在 IEEE MoDRE 工作坊上系统评估了 ChatGPT 从需求文档生成 UML 序列图的能力，发现需求质量对生成结果的完整性和正确性有显著影响[5]。

在测试自动化领域，Celik 与 Mahmoud（2025）在 *Machine Learning and Knowledge Extraction* 期刊上综述了 LLM 在自动测试用例生成中的四类方法（提示工程、反馈驱动、微调/预训练、混合方法）及效果对比[6]。Wang 等人（2024）在 arXiv 发表的综述则覆盖了 102 项研究，将测试用例准备和程序修复确定为 LLM 在测试领域最具代表性的两项任务[7]。

在文档生成领域，Venigalla 与 Chimalakonda（2025）提出的 DocFetch 框架可利用多层提示从多种软件制品中生成结构化文档，API 相关文档的 BLEU-4 得分约 43%[8]。

在游戏开发这一垂直领域，2025 年同样涌现了多项实证研究：IAR 期刊发表的 Unity 案例研究考察了 LLM 在学生短期游戏开发中的模块化任务辅助效果，发现 LLM 在模板化模块上显著缩短开发周期，但在性能关键代码上仍需人工监督[9]。SE4GAMES 2025 工作坊论文系统评估了 o3、ChatGPT-4o、Gemini 2.5 Pro 在 Unity/Unreal/Godot 三大引擎上的 30 个技术问题解答质量，发现 o3 显著优于其他模型，但完整性是所有模型的共同短板[10]。

综上所述，AI 已覆盖软件全生命周期的各个阶段——需求捕获与验证、架构设计与 UML 生成、代码生成与审查、测试用例生成与缺陷预测、文档自动生成——且各阶段均有可落地的学术方案与商业工具。本报告将结合《空箱》项目的 P1-P6 主过程与 S1-S3 支撑过程，识别 5 个 AI 可显著提效的关键环节，并逐一进行场景匹配分析。

2. 《空箱》项目软件过程概览

《空箱》是一个基于 Unity 2022.3 + C# + QFramework 的 2D 游戏项目，采用「保主干、轻流程、重反馈」的剪裁策略，参考 ISO/IEC 12207、15504、33001 定义了如下过程体系：

类别	编号	过程名称	核心活动
主过程	P1	立项与范围定义	明确迭代目标、拆解 MVP、形成任务清单
	P2	需求与规则建模	提炼核心实体、定义状态约束、设计关卡 JSON 结构
	P3	架构与详细设计	定义模块边界、类职责划分、流程编排器设计、事件驱动机制
	P4	实现与自测	脚本编写、场景联调、日志定位、回归验证
	P5	集成与验收	多模块集成、主路径/异常路径验证
	P6	迭代复盘与演进	记录高频问题、形成优化计划、同步文档
支撑过程	S1	配置与版本管理	Git 管理、目录结构约定、提交跟踪
	S2	质量保证与验证	场景联调、回归清单、双轨检查（主流程+异常流程）
	S3	文档与知识沉淀	关卡数据格式约定、流程配置说明、开发约定维护

3. AI 应用场景匹配分析

场景一：P3 架构与详细设计 → AI 驱动架构设计 + 自动化 UML 建模 (★ 重点)

痛点分析：P3 是《空箱》项目中最依赖经验的环节。基于 QFramework (Model/System/Command) 进行模块边界划分、设计 `GameFlowController` 的流程编排步骤模型 (LoadLevel/Dialogue/Wait/Signal)、定义事件驱动更新机制——这些决策直接影响后续所有阶段的效率。在小团队 GameJam 场景下，架构设计往往依赖个人经验，缺乏系统性的方案对比与验证。

AI 技术/工具匹配：

能力	推荐工具/框架	说明
架构方案生成与评审	GPT-5.5 + 结构化 Prompt	输入项目约束 (QFramework、关卡驱动、事件机制)，AI 生成多套架构方案并对比优劣
UML 图自动生成	PlantUML + DeepSeek-V4-Pro Claude Code Agent	从自然语言描述生成 Class Diagram、Sequence Diagram；Ferrari 等人 (2024) 的研究表明序列图最接近人工质量，但需求中存在歧义时会显著降低完整性和正确性[5]
设计模式推荐	Codex	在编写架构代码时，AI 根据上下文推荐合适的模式（如 Command 模式、Observer 模式）

学术支撑：

- Tagliaferro 等人 (IEEE ICSA-C 2025) 评估了 LLM 从非正式自然语言规格生成 UML 组件图的能力，开发了量化指标将 LLM 生成图与专家绘制的地面真值进行对比。结论是 LLM 在架构设计自动化方面显示出前景，但当前仍缺乏部署于真实场景所需的准确度[4]。
- Ferrari 等人 (IEEE MoDRE 2024) 测试了 ChatGPT 从 28 份不同领域的需求文档生成 UML 序列图，发现生成的图大体符合 UML 标准且可理解，但完整性和正确性在需求包含异味（歧义、不一致）时显著下降[5]。

预期价值：

- 架构设计阶段可探索的方案数量提升 3-5 倍，降低遗漏更优方案的风险
- 对于 `GameFlowController` 这类核心流程编排器，AI 可辅助生成步骤模型定义与状态转移验证
- AI 在此环节定位为「方案孵化器」而非「最终决策者」，需人工专家审核

场景二：P4 实现与自测 → AI 代码生成与智能审查

痛点分析： P4 涉及核心逻辑脚本、流程控制、关卡加载等大量 C# 编码工作。在小团队中，代码审查覆盖不足，潜在的缺陷（如空引用、反序列化异常）往往要到联调阶段才能发现。Unity API（如 Addressables、UniTask）的使用方式复杂，查阅文档占用较多时间。

AI 技术/工具匹配：

能力	推荐工具/框架	说明
C# 代码生成与补全	Codex	AI 根据项目上下文生成 C# 脚本，特别适用于模板化代码和 Unity API 调用
智能代码审查	DeepSeek-V4-Pro + Claude Code Agent	自动检测潜在空引用、资源泄漏、性能问题
代码重构	DeepSeek-V4-Pro + Claude Code Agent	识别耦合点并推荐重构方案，对齐 QFramework 架构约束
多 Agent 自主开发	CodePori（学术框架）	Rasheed 等人（2024）提出的多 Agent LLM 系统，可自动化完整 SDLC，HumanEval pass@1 达 87.5%，从业者满意度 91%[11]

学术支撑：

- Science China Information Sciences（2025）的综述汇总了多项研究，指出 LLM 辅助开发可将任务完成速度提升 55.8%，代码生成准确率较传统工具有 18.1%-25% 的提升[3]。
- 在游戏开发垂直领域：IAR Journal（2025）的 Unity 案例研究发现，LLM 在模板化模块（UI 脚本、基础交互逻辑）上可大幅缩短开发周期，但在性能敏感代码和跨模块一致性上仍需人工介入[9]。

风险提示： AI 生成的代码仍需经过「人工审查 → Editor 联调」的验证闭环，对核心逻辑（如关卡加载、信号触发）不应盲目信任 AI 输出。

场景三：P2 需求与规则建模 → AI 辅助需求捕获与游戏规则形式化

痛点分析：P2 的职责是将「玩法想法」转化为可编码的规则（实体定义、状态约束、关卡 JSON 结构）。在 GameJam 场景中，需求往往是口头讨论的结果，缺乏结构化的文档沉淀，导致后续开发中需求理解偏差。关卡 JSON 结构（LevelData / EntityData / EndPoints）的设计迭代也较频繁。

AI 技术/工具匹配：

能力	推荐工具/框架	说明
需求文档自动生成	GPT-5.5	从会议记录或口头描述生成结构化需求文档
需求一致性验证	LLM + 规则检查 Prompt	自动检测「实体定义」与「关卡 JSON 结构」之间的一致性
游戏规则形式化	结构化 Prompt + JSON Schema 生成	将自然语言玩法描述转化为规范的 JSON Schema

学术支撑：

- Cheng 等人（2025）在 *Software: Practice and Experience* 的系统文献综述（238 篇文章）指出：GenAI 在需求工程中的研究分布不均——分析（30.0%）和获取（22.1%）最受关注，管理（6.8%）则被忽视；超 90% 的研究处早期阶段，仅 1.3% 达生产级集成[2]。
- 该综述同时指出，评估体系碎片化、基准数据稀缺、幻觉问题（63.4% 的研究提及）是制约产业落地的核心瓶颈[2]。

预期价值：

- 需求文档化效率提升，降低因口头传递导致的信息衰减
- 自动检测数据模型与规则定义之间的不一致，减少反序列化异常
- 为关卡数据 JSON 提供版本化 Schema 约束

场景四：P5/S2 测试与质量保证 → AI 测试用例生成与智能缺陷预测

痛点分析：《空箱》项目当前的验证以人工场景联调为主（主路径+异常路径双轨检查）。关卡加载、对话推进、信号触发等流程的回归测试依赖手动操作，效率偏低。AI4SE 2024 年度报告显示，超 60% 的企业功能缺陷率降低了 20%-39%[1]。

AI 技术/工具匹配：

能力	推荐工具/框架	说明
测试用例自动生成	GPT-5.5 + Unity Test Framework	从需求描述生成 PlayMode/EditMode 测试脚本
预测性缺陷检测	基于 LLM 的缺陷预测模型	基于历史 Bug 模式预测高风险模块
游戏单元测试生成	微调 Code Llama	针对 Unity C# 的单元测试自动生成，有效扩展测试覆盖[12]

学术支撑：

- Celik 与 Mahmoud (2025) 综述了 LLM 在自动测试用例生成中的应用，将现有方法分为四类：提示工程、反馈驱动、微调/预训练和混合方法，指出混合方法在覆盖率和正确性上表现最优[6]。
- Wang 等人 (2024) 的综述 (102 项研究) 确认测试用例准备和程序修复是 LLM 在测试领域最具代表性的任务，多 Agent 协作架构将成为趋势[7]。
- 在游戏测试领域，已有研究 (Procedia Computer Science, 2024) 对 Code Llama 进行微调，专门用于 Unity C# 代码的单元测试生成，证明该方法可有效扩展游戏项目的测试覆盖率[12]。

预期价值：

- 系统性地枚举关卡数据异常、输入未配置、信号未触发等边界场景
- 将缺陷检测从「联调发现」前移至「代码提交时」
- 弥补小团队测试资源不足的问题

场景五：S3 文档与知识沉淀 → AI 辅助文档生成与知识维护

痛点分析： S3 要求维护关卡数据格式约定、流程步骤配置说明和开发约定。在小团队快速迭代中，文档更新滞后于代码变更，知识衰减严重。《空箱》的关卡 JSON 结构、流程步骤模型等关键设计决策缺乏系统化演进记录。

AI 技术/工具匹配：

能力	推荐工具/框架	说明
代码→文档自动生成	DeepSeek-V4-Pro + Claude Code Agent	从代码和注释生成 API 文档、数据结构说明
变更日志自动整理	LLM + Git Diff 分析	将 Git 提交记录转化为语义化的变更记录
知识库维护	LLM + Markdown 结构化输出	将散乱的开发笔记整理为结构化的开发约定

学术支撑：

- Venigalla 与 Chimalakonda (2025) 的 DocFetch 框架证明了从多源软件制品（代码、注释、提交信息）生成结构化文档的可行性，API 相关文档 BLEU-4 得分约 43%[8]。
- Rasheed 等人 (2024) 的 CodePori 多 Agent 系统已验证了从需求到代码到文档的全自动化可行性，从业者满意度达 91%[11]。

预期价值：

- 文档更新伴随开发过程持续生成，而非「事后补课」
- Git 提交历史自动转化为语义化变更记录
- 降低知识因人员流动而流失的风险

4. 场景匹配总结矩阵

序号	匹配过程	AI 应用方向	核心工具/框架	学术支撑	优先级
1	P3 架构与详细设计	架构方案生成、UML 建模、设计模式推荐	GPT-5.5 + PlantUML、Codex	[4][5]	★★★ (用户痛点)
2	P4 实现与自测	代码生成、智能审查、多 Agent 开发	Codex、DeepSeek-V4-Pro + Claude Code Agent	[3][9][11]	★★★★
3	P2 需求与规则建模	需求文档生成、规则形式化、一致性验证	GPT-5.5 + JSON Schema	[2]	★★☆
4	P5/S2 测试与质量保证	测试用例生成、缺陷预测、游戏单元测试	LLM + Unity Test Framework	[6][7][12]	★★☆
5	S3 文档与知识沉淀	文档自动生成、变更日志整理	Claude Code、DocFetch	[8][11]	★☆☆

5. 后续工作建议

基于以上 5 个场景的匹配分析，建议在第二阶段「软件过程改进方案设计」中：

- 以 **P3 (架构设计)** 为改进重点，设计 AI 参与架构评审和 UML 生成的具体流程节点与质量门禁；
- P4 (实现与自测)** 纳入 **AI 代码审查**作为质量门禁，定义 AI 审查通过标准；
- 选取 **P3 或 P4** 作为第三阶段原型验证场景，使用真实 AI 工具执行并收集对比数据。

参考文献

[1] 中国信息通信研究院人工智能研究所. 《AI4SE 行业现状调查报告（2024年度）》[R]. 2024.

- 联合发布：中信银行、阿里云、华为云、腾讯、百度、字节、360、软通动力、Testin云测等 30+ 机构；有效问卷 1,813 份
- 36氪报道：<https://36kr.com/p/3261250178236164>
- Google Scholar: <https://scholar.google.com/scholar?q=AI4SE+行业现状调查+2024+中国信通院>

[2] Cheng, H., Husen, J.H., Lu, Y., Racharak, T., Yoshioka, N., Ubayashi, N. & Washizaki, H. (2025). Generative AI for Requirements Engineering: A Systematic Literature Review. *Software: Practice and Experience*, Wiley. (系统分析 238 篇文献，2019–2025 年)

- arXiv: <https://arxiv.org/abs/2409.06741>
- DOI: <https://doi.org/10.1002/spe.70029>
- Google Scholar: <https://scholar.google.com/scholar?q=Generative+AI+Requirements+Engineering+Systematic+Literature+Review+Cheng+2024>

[3] Science China Information Sciences. (2025). A Survey on Large Language Models for Software Engineering.

- DOI: <https://doi.org/10.1007/s11432-025-4670-0>
- Springer: <https://link.springer.com/article/10.1007/s11432-025-4670-0>
- Google Scholar: <https://scholar.google.com/scholar?q=survey+large+language+models+software+engineering+2025>

[4] Tagliaferro, A., Corbo, S. & Guindani, B. (2025). Leveraging LLMs to Automate Software Architecture Design from Informal Specifications. *IEEE ICSA-C 2025*, pp. 291–299. Politecnico di Milano.

- DOI: <https://doi.org/10.1109/ICSA-C65153.2025.00049>
- Semantic Scholar: <https://www.semanticscholar.org/paper/Leveraging-LLMs-to-Automate-Software-Architecture-Tagliaferro-Corbo/1d47cea948318479b167439781f1bc876851aff7>
- Google Scholar: <https://scholar.google.com/scholar?q=Leveraging+LLMs+Automate+Software+Architecture+Design+Informal+Specifications>

[5] Ferrari, A. et al. (2024). Model Generation with LLMs: From Requirements to UML Sequence Diagrams. *IEEE REW 2024 / MoDRE Workshop*.

- arXiv: <https://arxiv.org/abs/2404.06371>
- Google Scholar: <https://scholar.google.com/scholar?q=Model+Generation+LLMs+Requirements+UML+Sequence+Diagrams+Ferrari+2024>

[6] Celik, A. & Mahmoud, Q. (2025). A Review of Large Language Models for Automated Test Case Generation. *Machine Learning and Knowledge Extraction (MAKE)*, MDPI, 7(3), 97.

- DOI: <https://doi.org/10.3390/make7030097>
- Google Scholar: <https://scholar.google.com/scholar?q=Review+Large+Language+Models+Automated+Test+Case+Generation+Celik+2025>

[7] Wang, J., Huang, Y., Chen, C., Liu, Z., Wang, S. & Wang, Q. (2024). Software Testing with Large Language Models: Survey, Landscape, and Vision. *arXiv:2307.07221v3*. (覆盖 102 项研究)

- arXiv: <https://arxiv.org/abs/2307.07221>
- Google Scholar: <https://scholar.google.com/scholar?q=Software+Testing+Large+Language+Models+Survey+Landscape+Vision+Wang+2024>

[8] Venigalla, A.S.M. & Chimalakonda, S. (2025). DocFetch — Towards Generating Software Documentation from Multiple Software Artifacts. *arXiv:2508.17719*.

- arXiv: <https://arxiv.org/abs/2508.17719>
- Google Scholar: <https://scholar.google.com/scholar?q=DocFetch+Generating+Software+Documentation+Multiple+Artifacts+2025>

[9] Exploring the Applicability of Modular Tasking in Short-Term Student Game Development with Large Language Models: A Unity Case Study. (2025). *IAR Journal*, Vol. 3, pp. 54–.

- DOI: https://doi.org/10.20736/iar.3.0_54
- Google Scholar: <https://scholar.google.com/scholar?q=Modular+Tasking+Student+Game+Development+LLM+Unity+Case+Study+2025>

[10] Assessing Frontier LLMs in Solving Game Development Problems: Preliminary Findings Across Three Game Engines. (2025). *SE4GAMES Workshop @ SBC 2025*.

- DOI: <https://doi.org/10.5753/se4games.2025.14869>
- Google Scholar: <https://scholar.google.com/scholar?q=Assessing+Frontier+LLMs+Game+Development+Problems+Three+Game+Engines+2025>

[11] Rasheed, Z., Waseem, M., Saari, M., Systä, K. & Abrahamsson, P. (2024). CodePori: Large-Scale System for Autonomous Software Development Using Multi-Agent Technology. *arXiv:2402.01411*.

- arXiv: <https://arxiv.org/abs/2402.01411>
- Google Scholar: <https://scholar.google.com/scholar?q=CodePori+Autonomous+Software+Development+Multi-Agent+2024>

[12] LLM-based methods for the creation of unit tests in game development. (2024). *Procedia Computer Science*, ScienceDirect.

- ScienceDirect: <https://www.sciencedirect.com/science/article/pii/S1877050924025158>
- Google Scholar: <https://scholar.google.com/scholar?q=LLM+based+methods+unit+tests+game+development+2024>

撰写日期: 2026 年 6 月 13 日